



pdf-tree — QA Evaluation Guide

Internal Technical Reference • Support Engineering

Field	Value
Document owner	DataClever AI — Engineering
Architecture designed by	Jonathan Potes, CTO
Audience	Junior AI / Software Support Engineers
System version	pdf-tree v0.1.1
Classification	Internal — Proprietary. Not for external distribution. See LICENSE.md.

1. Purpose

This guide equips a Support Engineer with the operational knowledge required to **manually validate the output of the pdf-tree pipeline** — a document-structure extraction system that converts a PDF into a hierarchical `tree.json`, the primary input for downstream RAG chunking.

You do not need to modify the codebase to use this guide. You need to be able to:

- Explain what each pipeline stage does and why it exists.
- Run the pipeline against a sample PDF and read its output correctly.
- Distinguish a genuine defect from documented, by-design behavior.
- Locate the evidence (logs, validation report, `tree.json` fields) that supports a ticket, before escalating to Engineering.

2. System Overview

pdf-tree extracts a semantic tree from a PDF document: chapters, sections, subsections, each carrying its own paragraphs and tables as structured content nodes. No LLM is involved anywhere in the pipeline — every output field is produced by a deterministic, traceable rule. This is the single most important fact for triage: **if an output looks wrong, the cause is a specific line of code, not model non-determinism.**

The system combines two independent signals:

Signal	Source	Gives you
Structure	The PDF's embedded bookmarks (Table of Contents)	Hierarchy levels, section titles, claimed page numbers
Content	Docling layout/content extraction	Paragraphs, tables, reading order, bounding boxes

3. Technology Glossary

3.1 Docling

Official project: <https://docling-project.github.io/docling/> — open-source document-understanding engine, originally developed by **IBM Research** and now hosted under the **LF AI & Data Foundation**. Source: github.com/docling-project/docling.

Docling performs layout analysis and table-structure recognition on the PDF using local machine-learning models. It runs entirely on-device (CPU or GPU) — **no external API call, no document content leaves the environment**. This is a material point for an enterprise deployment handling proprietary or client manuals: there is no third-party inference endpoint in the extraction path.

3.1.1 Architecture — how Docling actually works Docling is a pipeline of specialized models, not a single black box. For a PDF input, the stages are:

1. **Backend parsing** — a PDF backend (this project uses PyPdfiumDocumentBackend, based on Google's PDFium) extracts the raw page content: text runs, embedded images, and low-level positioning, without yet interpreting layout meaning.
2. **Layout analysis model** — a vision model classifies every detected region of the page into a semantic category: text, section_header, table, picture, caption, page_header, page_footer, footnote, list_item, etc. This is what lets Docling tell a real body paragraph apart from a running header, a footnote, or a table caption — a plain text extractor cannot make this distinction.
3. **Table Structure Recognition (TableFormer)** — for every region classified as table, a dedicated model reconstructs the actual row/column grid, including merged cells, **even when the source PDF has no visible gridlines**. This project renders that structure into a markdown grid (`_render_table_markdown()` in `docling_extract.py`) instead of Docling's default placeholder.
4. **Reading order resolution** — regions are ordered into a single coherent sequence that matches how a human would read the page, correctly handling multi-column layouts. This produces the `reading_order` index this project depends on for section-content assignment (§4.2).
5. **Optional OCR** — for scanned/image-only pages (disabled in this project's configuration, `do_ocr = False`, since target documents are text-native PDFs).
6. **Document assembly** — all of the above is merged into one structured `DoclingDocument` object (a tree of texts, tables, pictures, groups, `key_value_items`), exportable to JSON, Markdown, or HTML.

Reference architecture diagram and full technical paper: Docling Technical Report (arXiv:2408.09869).

3.1.2 How this project uses it Integration point: `src/tree_builder/docling_extract.py`. It is an **optional** dependency — the pipeline degrades gracefully (structure only, no content assignment) if Docling is not installed (`DoclingExtractionEngine.is_available`).

Minimal Docling usage (standalone reference – not this project's exact call site)

```
from docling.document_converter import DocumentConverter
```

```
converter = DocumentConverter()
result = converter.convert("manual.pdf")
```

```
print(result.document.export_to_markdown())    # human-readable
doc_dict = result.document.export_to_dict()    # structured – what this project consumes
```

This project's actual usage (`DoclingExtractionEngine.extract`, `docling_extract.py`) adds two production-grade layers on top of the minimal call above:

- **Windowed extraction** — Docling processes the PDF in overlapping page windows (default `window_size=60`, `window_overlap=6`) instead of one call over the whole file, so a several-hundred-page manual does not need to be held in memory at once. Duplicate content at window boundaries is trimmed in `_trim_overlap_pages`.

- **Typed conversion** — Docling’s raw dict output is converted into this project’s own immutable dataclasses (DoclingTextBlock, DoclingTable, §4.1) so the rest of the pipeline never touches Docling’s internal schema directly.

Heading levels detected by Docling’s `HeadingHierarchyOptions` are used **only** as the fallback signal for the synthetic-TOC path (§4, Stage 4) — on the normal path, hierarchy comes from the PDF’s own embedded bookmarks, not from Docling.

3.1.3 Why Docling instead of a plain text-splitter (LangChain, etc.) A common alternative approach — `PyPDFLoader` / `pdfplumber` + `LangChain`’s `RecursiveCharacterTextSplitter` — extracts a flat text stream from the PDF and cuts it into fixed-size character windows. It is faster to set up, but it discards exactly the information this project’s tree structure depends on:

Concern	Plain text-split (LangChain-style)	Docling (this project)
Layout awareness	None — multi-column pages are frequently read out of order, interleaving column 1 and column 2 text mid-sentence.	Layout model classifies and orders regions correctly, including multi-column pages.
Chunk boundaries	Arbitrary — a fixed character/token count, blind to sentence, paragraph, or section boundaries. Routinely splits mid-sentence or mid-table.	Boundaries follow real document structure (<code>reading_order</code> , heading labels) — this project chunks along actual section/paragraph units, never an arbitrary character count.
Headers/footers/footnotes	Pollute the text stream indiscriminately — a running page header can appear mid-chunk, corrupting retrieval relevance.	Classified and separable by label (<code>page_header</code> , <code>page_footer</code> , <code>footnote</code> vs <code>text</code>) — this project’s pipeline reads only content-bearing labels.
Tables	Collapse into unstructured whitespace-separated text, usually unusable for retrieval.	Reconstructed as a real row/column grid via <code>TableFormer</code> , rendered to clean markdown (§5.6 — a table with an empty placeholder is a defect, not expected behavior).
Section identity	None built-in — a splitter has no notion of “this chunk belongs to Chapter 3, Section 2.”	Every content block carries a page number and reading-order position that this pipeline anchors to a specific <code>section_id</code> (§4).
Traceability	Typically lost after splitting.	Each block retains its bounding box (<code>BoundingBox</code>) and page number — enables tracing a retrieved chunk back to its exact location in the source PDF, useful for citations and audits.
Data locality	Depends on the loader; some hosted parsers call external APIs.	Fully local inference (CPU/GPU) — no data leaves the environment (§3.1 opening paragraph).

In short: a plain text-splitter treats a PDF as an undifferentiated string. Docling treats it as a structured document with typed regions, a reading order, and reconstructable tables — which is the only way this project’s tree-building approach (bookmarks + content matched by `reading_order`, §4.2) is possible at all. Chunking by character count is a reasonable fallback for unstructured plain text; it is the wrong tool once the source has real hierarchical structure to preserve, which is the case for every technical manual this system targets.

3.2 fitz (PyMuPDF)

fitz is the Python binding for **PyMuPDF** (`import pymupdf as fitz`), used for low-level PDF operations: page counting, embedded image extraction, page rendering, and — critically — reading the **Table of Contents**.

3.3 TOC (Table of Contents)

The TOC is the PDF's **embedded bookmark structure** — the navigation panel a PDF viewer shows on the side. It is not the printed “Table of Contents” page; it is structured metadata baked into the file: a list of (level, title, page_number) tuples.

```
# src/tree_builder/fitz_toc.py
toc = doc.get_toc()
# e.g. [(1, "1. Introduction", 3), (2, "1.1 Scope", 4), (1, "2. Installation", 8), ...]
```

If a PDF ships with no embedded TOC, the pipeline builds a **synthetic** one from Docling's detected section_header blocks (`src/tree_builder/synthetic_toc.py`). This path is inherently less reliable — there is no external ground truth to anchor against — and is flagged in the output via `structural_source`: “inferred”.

4. Pipeline Architecture

Orchestrated end-to-end by `run_pipeline()` in `src/pipeline/pipeline.py`. Ten stages, executed strictly in order:

PDF

```
|
|-- 1. fitz_toc          -> embedded bookmarks + total page count
|-- 2. bookmark_sanity   -> structural integrity pre-check (hard-aborts on disorder)
|-- 3. docling_extract    -> text blocks + tables + heading levels (optional)
|-- 4. synthetic_toc     -> fallback TOC if no embedded bookmarks exist
|-- 5. toc_classification -> discards front-matter noise, sets aside back-matter
|-- 6. toc_resolution    -> numbering scheme detection, page-offset calibration,
                        title verification against extracted text
|-- 7. section_matcher   -> assigns content blocks to sections by reading order
|-- 8. tree_export        -> serializes to tree.json
|-- 9. tree_validator     -> 3 automated checks: Coverage, Precision, Structure
|-- 10. image_extraction -> embedded images, heuristically filtered, mapped to section
```

Each stage is an isolated module under `src/tree_builder/`. Failures are never silent — every stage's wall-clock time is captured in `PipelineResult.stage_times`, giving you a precise point of failure before you ever open a debugger.

4.1 Module Map

Module	Responsibility
<code>src/models/extraction.py</code>	Immutable typed entities: <code>BoundingBox</code> , <code>DoclingTextBlock</code> , <code>DoclingTable</code> , <code>DoclingDocument</code>
<code>src/pipeline/pipeline.py</code>	Orchestrator — <code>run_pipeline()</code> , <code>PipelineResult</code> , embedded-image extraction
<code>src/tree_builder/fitz_toc.py</code>	PyMuPDF wrapper: TOC, page count, page rendering
<code>src/tree_builder/bookmark_sanity.py</code>	Pre-flight integrity check on raw bookmarks
<code>src/tree_builder/docling_extract.py</code>	Docling engine, windowed extraction + merge
<code>src/tree_builder/synthetic_toc.py</code>	Fallback TOC inferred from Docling headings

Module	Responsibility
src/tree_builder/toc_classification.py	Regex-based front-matter / back-matter filtering — no LLM
src/tree_builder/page_numbering.py	Numbering-scheme detection (arabic / roman / alphanumeric)
src/tree_builder/title_verification.py	Fuzzy match (RapidFuzz) of a title against text near its claimed page
src/tree_builder/toc_resolution.py	Combines numbering + verification into a resolved physical page + offset
src/tree_builder/section_matcher.py	Assigns content to sections by reading order
src/tree_builder/tree_export.py	Serializes matched sections to tree.json
src/validation/tree_validator.py	Coverage / Precision / Structure checks

4.2 Design Rationale — Why Execution Order Matters

These are engineering decisions, not incidental ordering — know them before filing a defect against behavior that is, in fact, intentional.

- **Stage 5 (classification) runs before Stage 6 (resolution).** Front-matter noise entries (“Abstract”, “List of Figures”) are typically among the first few bookmarks — exactly where page-offset calibration draws its sample. Filtering them out first prevents noise from corrupting the calibration.
- **Content assignment (Stage 7) uses reading_order, never page_no, for text blocks.** If three subsections share one physical page, page-based assignment would route all of that page’s content to whichever section is listed first, starving the others (`node_count: 0`). Docling’s global sequential `reading_order` correctly disambiguates section boundaries within a shared page. **Tables are the one exception** — Docling does not assign them a `reading_order`, so they fall back to page-based assignment (`src/tree_builder/section_matcher.py::_assign_content`), a known, accepted coarser path.
- **A bookmark that fails title verification is never dropped.** It is still built into a section, but marked `flagged_for_review: true`. Design principle: a misplaced-but-visible section is always preferable to silently vanished content.

5. Evaluation Procedure

5.1 Environment Setup

```
uv sync
uv run streamlit run streamlit_app/app.py
```

Evaluation walkthrough across the app’s pages:

1. **Extraction** — upload the PDF, run the pipeline with Docling enabled.
2. **Tree** — navigate the generated hierarchy side-by-side with the source PDF.
3. **Metrics** — review the automated validation report and per-stage timings.
4. **Images** — review extracted images grouped by section.
5. **Export** — download `tree.json`, `images_v1.json`, `bookmarks.json` for offline inspection.
6. **View** — re-load a previously exported `tree.json` without re-running the pipeline (useful to diff two runs).

5.2 Where the Evaluation Evidence Lives

Before filing or escalating a ticket, gather evidence from these sources — in this order:

Evidence	Location	Notes
Validation report	<code>PipelineResult.validation -> ValidationReport.to_dict()</code> (<code>src/validation/tree_validator.py</code>) surfaced in the Metrics page	First stop. status: FAIL includes concrete offending items (misplaced_blocks, level_mismatches, orphan_sections) — up to 20 examples per check, not just a boolean.
Stage timings	<code>PipelineResult.stage_times</code> (dict, one float per stage), Metrics page	Identifies which stage a slowdown or partial failure originates in.
Console log stream	<code>setup_logger()</code> in <code>streamlit_app/services/pipeline_service.py</code> 54	Currently console-only (<code>logging.StreamHandler</code> , format <code>LEVEL logger.name message</code>) — no file handler is attached. When running via <code>uv run streamlit run ...</code> , this stream lands in the terminal that launched Streamlit; it is not persisted across sessions.
tree.json	Downloaded from the Export page	Full structural record — see §5.4 for the fields that matter.
excluded_toc_entries	<code>PipelineResult.excluded_toc_entries</code> — not written to <code>tree.json</code> , inspect via a custom script or add to the Export page	Bookmarks classified as front/back matter and set aside — verify none of these were a real content section misclassified by <code>toc_classification.py</code> .

Gap to flag to Engineering: there is currently no persistent, timestamped log file for pipeline runs — only an ephemeral console stream. For any ticket requiring audit trail (a run that can no longer be reproduced live), request a `logging.FileHandler` be added at the `setup_logger()` call site (`streamlit_app/services/pipeline_service.py:47`) or that `run_pipeline()`'s `logger` argument (`src/pipeline/pipeline.py:168`) be pointed at a file sink before escalating a non-reproducible issue. Until that exists, capture the console output manually (`uv run streamlit run streamlit_app/app.py 2>&1 | tee run.log`) for any session you intend to reference later.

5.3 Automated Validation Checks

Three checks run on every pipeline execution (`src/validation/tree_validator.py`). Overall status is PASS only if all three pass.

Check	What it verifies	Typical failure cause
Coverage	Every Docling content block appears in exactly one section (<code>orphaned_count == 0</code> , no duplicates)	A block wasn't captured by any section's reading-order range — check <code>expected_front_matter</code> accounting in <code>src/pipeline/pipeline.py:383–387</code>
Precision	Every node's <code>page_no</code> falls within its section's [<code>page_start</code> , <code>page_end</code>] ($\pm$ <code>page_tolerance</code>)	Page-offset miscalibration in Stage 6 (<code>toc_resolution.py</code>)

Check	What it verifies	Typical failure cause
Structure	Hierarchy matches the bookmarks — level agreement, no orphans, no cycles, no missing parents	Bug in <code>section_matcher._build_hierarchy</code> , or corrupt bookmarks in the source PDF

Important: a PASS verdict is a structural/mechanical guarantee, not a semantic one. It does not by itself confirm the tree is *meaningfully* correct — manual review per §5.4 remains required even on a passing run.

5.4 Manual Structural Review Checklist

For each section in `tree.json`, cross-checked against the source PDF opened side-by-side:

- ☐ **hierarchy_level** matches the bookmark’s real nesting level in the PDF.
- ☐ **parent_section_id / child_sections** are structurally coherent (e.g. “1.2 Scope” is a child of “1. Introduction”, not of an unrelated preceding section).
- ☐ **page_start / page_end** — open those exact pages; confirm the section’s content genuinely starts and ends there. Most sensitive check to page-offset miscalibration.
- ☐ **hierarchy_path** (e.g. /Chapter 1/Section 2) reflects the true path from the root.
- ☐ **No gaps, no duplicates** — every section visible in the PDF’s real index is present exactly once.
- ☐ **flagged_for_review: true** — always inspect manually. `verification_score` and `verification_reason` explain why the title match near the claimed page fell below threshold; these are the highest-probability candidates for misplacement.
- ☐ **structural_source** — if “inferred” rather than “toc”, the source PDF had no embedded TOC and the hierarchy was inferred from heading styles. Apply extra scrutiny; this path carries a materially higher structural error margin by design.

5.5 Embedded Image Extraction

`_extract_embedded_images()` (`src/pipeline/pipeline.py:444–536`) applies five heuristic filters before accepting an image, to suppress decorative icons, header/footer logos, and divider rules:

#	Filter	Threshold
1	Minimum pixel dimensions	≥ 80px on each side (hard floor, regardless of the configured <code>min_image_px</code>)
2	Minimum bounding-box area	≥ 2% of page area
3	Header zone exclusion	Fully within top 5% of page height
4	Footer zone exclusion	Fully within bottom 5% of page height
5	Aspect ratio	Rejects ratios > 15:1 (thin decorative strips)

Checklist: - [] Are relevant diagrams/screenshots being dropped by the filters? Consider whether `min_image_px` needs adjustment for this document class. - [] Are logos/icons incorrectly passing the filters? May indicate the area/aspect-ratio thresholds need document-specific tuning. - [] Is `section_id` (`_map_images_to_sections`, `src/pipeline/pipeline.py:125–155`) assigned to the correct section? Images are mapped by `page_no` to the deepest section containing that page — verify against pages shared by multiple sibling sections.

5.6 Additional Field-Level Checks

- ❑ **numbering_scheme** (`arabic / roman / alphanumeric / unknown`) per section — a document with roman-numeral front matter transitioning to arabic body pagination should show that transition at the correct section boundary.
 - ❑ **offset_applied** — the delta between claimed TOC page and resolved physical page. Unexplained large swings between adjacent sections suggest miscalibration.
 - ❑ **Table nodes** (`node_type: "table"`) — `canonical_text` should contain the rendered markdown grid, not an empty `[Table RxC]` placeholder. An empty placeholder means Docling returned no cell data for that table.
 - ❑ **sanity_report** — if the pipeline aborted outright, the cause (out-of-order bookmarks — a hard failure) is recorded here.
-

6. Out of Scope — Do Not File as Defects

The following are documented, intentional design boundaries for this release (Phase 1 of the product roadmap):

- No LLM/VLM is used at any stage. All output is deterministic (fitz + Docling + rule-based logic + fuzzy matching).
 - Without an embedded TOC, hierarchy is a best-effort inference from heading styles (`structural_source: "inferred"`) — higher error margin is expected and documented behavior, not a bug.
 - Tables are assigned to sections by page number, not reading order — a known, accepted coarser fallback (§4.2).
 - Features belonging to later roadmap phases (RAG pipeline merge, MCTS agent) are not present in this version.
-

7. Escalation Guidance

Escalate to Engineering when:

- A validation check fails (`status: FAIL`) on a document where manual inspection (§5.4) confirms the structure is genuinely wrong — attach the failing check's `details` block and the source PDF.
- A pipeline run raises an unhandled exception (`PipelineResult.error` populated) that is not one of the two documented hard-abort conditions (out-of-order bookmarks, or zero usable TOC/headings).
- Behavior does not match §6 (Out of Scope) and is reproducible.

Do not escalate:

- `flagged_for_review: true` sections alone — this is expected output for low-confidence matches, not a failure.
 - Structural degradation on a document with `structural_source: "inferred"` — expected, unless it falls meaningfully outside normal tolerance for that class of document.
-

DataClever AI — Internal Use Only. Copyright (C) 2026 DataClever AI. All rights reserved. See `LICENSE.md`.